PiTrace Workbench: An Application That Emerges From the Agent

Team 1286 — KDD Cup 2026 Creative Track

Abstract

PiTrace is an artifact-native workbench for complex data analysis in which an autonomous agent and a human analyst share the same evolving task state. The central idea is not to bolt an assistant onto a dashboard, but to let the application emerge from the agent's own work: source descriptions become a discovery graph, solve phases become an answer thread, uncertainty becomes a human decision wall, accepted guidance becomes reusable memory, and the final answer is published only through a contract-gated deterministic pipeline. The system is built around three contributions. First, it provides a **human-agent workbench** for defensible analysis: the machine performs scalable source inspection, planning, code generation, validation, review, repair, and publishing, while the human is invited at the narrow points where judgment is needed. Second, it treats analytical correctness as reusable **skills**, not per-task pipelines: the solver uses 17 task-agnostic Markdown skills (about 2,500 lines) developed through a leak-safe, gold-supervised, human-gated learning loop. Third, it separates reasoning from publication: each generated `solution.py` is the deterministic harness produced by that task's agentic loop, encoding the discovered contract, evidence, joins, metrics, and output shape, while the publisher emits only contract-supported columns. Across validation on all 111 KDD-shared Phase 1 and Phase 2 input tasks, plus seeded interactive workspaces and a released subset of solver traces, PiTrace demonstrates multi-source joins, domain adaptability, video-derived evidence, multilingual records, repair from feedback, and principled refusal. We used Qwen 3.5/3.6-family models behind a swappable OpenAI-compatible endpoint served through OpenRouter or local LlamaCpp; stronger compatible models can be substituted without changing solver code. The reliability claim is architectural: it comes from the workbench, reusable skills, phase prompts, validation, repair, and publish gate, not from relying on a particular model choice.

1. Problem and Motivation

Real analytical work rarely looks like a single-table question. Analysts and domain experts combine CSV files, JSON records, databases, documents, video, images, and local business assumptions. They must determine which sources are relevant, interpret ambiguous terms, choose joins and metrics, verify that an answer is actually supported, and preserve lessons for related future questions.

The Creative Track is valuable because it asks for systems that look closer to this work than a leaderboard-only solver. A strong data agent should not merely answer when it is lucky; it should expose what it understood, what evidence it used, what it could not ground, and where a human can improve the result. The primary user for PiTrace is therefore a working analyst, auditor, or domain expert who must produce a defensible answer from messy, multi-source data. They need scale and accountability: evidence lineage, an output contract, a deterministic code path, correction hooks, and an honest abstention path.

This includes professional teaching and decision-support settings where the data is mostly prose. In the medical problem-based learning (PBL) workspace, the user is closer to a clinician-educator or medical learner than a spreadsheet analyst: they need to apply a chronic coronary disease teaching module, therapy appendices, and patient narratives to open-ended case questions. Existing benchmark-style agents are weak at this because there is no table schema, no fixed answer column, and no single lookup. The task is to understand the local teaching corpus, identify what the case has and has not established, produce a reviewable structured answer, and let the human ask for clearer justification when the first answer is under-explained.

Our premise is that no autonomous model should be expected to resolve every under-specified analytical question alone. The machine can perform the tedious and scalable work, but the remaining judgment points — ambiguous requirements, source interpretation, metric definition, and error correction — need a surface where the human and the agent can reason over the same state. PiTrace is built around that surface.

2. Why Existing Approaches Fall Short

One-shot LLM answering is not a data agent. It does not reliably perceive the state of a data environment, choose tools, execute deterministic analysis, verify the result, or repair itself from feedback. It also hides commitments such as row grain, output columns, join policy, metric definitions, and abstention criteria. When evidence is missing, it tends to produce fluent uncertainty rather than a reviewable refusal.

Simple tool-calling agents improve on one-shot answering but often collapse planning, evidence, code, review, and presentation into one opaque conversation. When they are wrong, the user sees a final answer or a transcript, not the structured state needed to decide whether the result is defensible.

Conventional dashboards have the opposite limitation. They expose data well, but they do not show what an autonomous agent is doing with that data — no evolving interpretation, no repair loop, no uncertainty state, and no durable memory.

PiTrace inverts this pattern. The app renders the artifacts and events the solver already writes: discovery state, contracts, evidence maps, plans, validation reports, review decisions, wall requests, feedback, memory, and final predictions. The human is not reading logs after the fact; the human and the agent work from one shared model of the task.

3. System Overview

PiTrace has three layers (Figure 1).

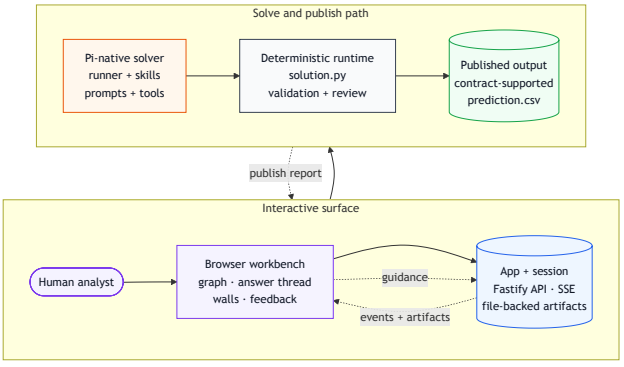


Figure 1 — System architecture: browser app, Fastify server, file-backed sessions, Pi solver, deterministic Python runtime, published output. A detailed version is included in Appendix I.

Browser workbench. The `app/` layer is a Fastify and vanilla-JS browser application with file-backed sessions. It provides workspace management, drag-and-drop ingestion, live discovery, source previews, answer threads, wall resolution, post-answer feedback, memory editing, and replayable event streams. It spawns the solver as a subprocess, tails its JSONL

event log, and forwards events to the browser over Server-Sent Events. It does not call an LLM or generate an answer itself.

Pi-native solver. The `solvers/pi/` layer owns the agentic reasoning and execution loop. Solver behavior is expressed through project-local Pi resources: `.pi/skills`, `.pi/prompts`, and `.pi/extensions`. The TypeScript runner stays thin: it sequences phases, enforces timeouts and repair caps, runs deterministic validation, and publishes. Secondary LLM needs are exposed as Pi tools using the current model/provider context, so model use remains visible through tool logs.

Internally, PiTrace is built from two Pi.dev agent specializations rather than one monolithic assistant. The app-facing **discovery engine** is launched by `app/src/server.js` through `npm run discover`; it runs the `/discover` prompt, loads the `discovery-orientation` skill, and writes real-time source-graph events through `kdd_discovery_emit`. The question-driven **answer engine** is launched through `npm run task` or `npm run feedback`; it runs solve, repair, review, feedback, and learning prompts, loads answer-contract/review/modality skills, and uses tools such as `kdd_discovery_query`, `kdd_request_human`, `kdd_prose_relation`, `kdd_prose_adjudicate`, and the app-only `kdd_solve_emit`. Both engines share the same Pi SDK worker, model/provider registration, project-local resource directory, run-context contract, artifact store, and event bridge. Their different behavior comes from phase-specific Pi resources and tool registration, not from a second hidden LLM service.

Deterministic Python runtime. The generated `solution.py` is the concrete outcome of one answer loop: the agent has already discovered the relevant sources, framed the contract, mapped evidence, and planned the computation, and the file encodes that task-specific approach as local deterministic Python. It is restricted to Python stdlib, `sqlite3`, and `pandas`; it must not call Pi, LLMs, network services, package installers, or shell commands. The runner validates the raw prediction and publishes only contract-supported columns to `published.csv` and the official `prediction.csv`.

The system runs two loops, expanded in Appendix I. The **discovery loop** is question-blind: it profiles the workspace, identifies sources and modalities, emits source descriptions, discovers relationships, forms source groups, and raises concerns. The **answer loop** begins with a natural-language question: it builds an answer contract, gathers evidence, writes a plan, generates deterministic code, executes it, validates the output, runs an independent no-gold review, repairs if needed, and then publishes or abstains.

The handoff between the two loops is also engineered as data, not as transcript memory. Discovery emits `discovery/stream.jsonl`; the compiler replays that stream into `discovery-summary.md`, `discovery/graph.yaml`, `discovery/passages.jsonl`, and `discovery/media-tables.jsonl`; answer phases then query those artifacts through a typed Pi tool before re-reading raw files. This lets the answer engine reuse the source graph, media rows, passages, and concerns discovered by the first engine while preserving the no-gold, question-specific answer contract.

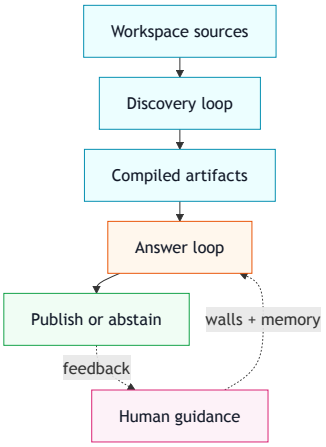


Figure 2 — Simplified two-loop workflow: discovery artifacts feed the answer loop, and human guidance enters only as bounded steering.

4. Core Contributions

4.1 Agent-Emergent Workbench

The interface is not a static dashboard with an agent attached. The agent emits structured events and artifacts, and the app turns those emissions into the user experience itself. Source descriptions become graph nodes, relationships become edges, solve phases become an answer timeline, wall requests become decision prompts, and accepted guidance becomes memory for future runs.

This is the submission's main interaction paradigm. A user can watch the agent come to terms with a workspace, inspect the source graph, ask a question, follow the answer lifecycle from contract framing through evidence gathering, plan selection, deterministic coding, and review, open the final CSV, and intervene when the agent needs judgment. The workbench is therefore both a product surface and an observability layer over the agent's real task state.

4.2 Human-in-Control: Walls, Feedback, and Memory

PiTrace supports two complementary intervention paths and a three-scope persistence layer. The first intervention path is live engagement while the solver is running; the second is a feedback continuation after an answer exists. Memory is separate from both paths: accepted guidance can be kept at platform, workspace, or question/task scope and compiled into later solve contexts.

Walls are agent-requested decisions. When the selected autonomy mode allows it, the solver can pause at question understanding, plan quality, or final prediction review and ask the human for a bounded decision. A wall request contains the decision needed, two or three options, the agent's proposed default, evidence references, and what will happen if no answer arrives. The human's resolution is written as an artifact and can be pinned into memory. Human input is treated as guidance, not source truth: generated code must still ground asserted facts in the task data.

Post-answer feedback handles the other common case: the agent was confident, published a mechanically valid answer, and the user notices the semantic error. The user adds a focused correction, and the solver continues from the existing answer context rather than restarting or editing the CSV directly. Our repair examples include corrections such as "select events with lowest-cost items, not lowest total event cost," after which the agent repaired the interpretation and changed the output.

Memory closes the loop. Platform memory applies across all workspaces and questions, workspace memory applies across questions in one uploaded corpus, and question/task memory applies to a specific answer thread. The app stores these notes as plain Markdown, compiles the applicable scopes into

`context/effective-memory.md`, and passes them to the next solve as guidance. They are not unchecked answer sources: generated code and published values must still be grounded in task data and pass validation and publish gates. Appendix E details the feedback and memory mechanics.

4.3 Reusable Skills Instead of Per-Task Pipelines

The central methodological choice is to move correctness out of per-task code and into reusable analyst skills. PiTrace ships 17 Markdown skills totaling about 2,500 lines. They are not narrow task recipes and are not tuned to the visible demo scenarios. Instead, they encode general procedures for recurring analytical failure modes: ambiguous row grain, unsupported columns, weak joins, hidden metric definitions, value normalization, prose fact recovery, media-derived evidence, no-gold review, and human escalation. Skills such as `question-interpretation`, `join-planning`, `metric-grain`, `aggregation-and-ties`, `value-normalization`, `prose-fact-search`, and `no-gold-review` encode decision procedures, red-flag checklists, and selection rules that a model executes through reflection.

The important generalization claim is negative as much as positive: the skills must not contain public task IDs, memorized answers, gold-derived facts, or hard-coded publish columns. They may describe generic source patterns or example syntax, but the actual task answer still has to be derived from the current workspace's contract, evidence, and deterministic code. This is what lets one library cover CSV/database joins, professional prose modules, video-derived thresholds, multilingual records, repair, and principled abstention without becoming a collection of scenario-specific shortcuts.

The per-task Python is deliberately disposable. It is regenerated from the contract, evidence, and plan for each run, then treated as a deterministic executor. The durable asset is the skill library: a reviewer can open the skills and inspect how row grain, join policy, tie handling, prose evidence, and abstention are supposed to work. In short, we write the analyst, not the analysis.

4.4 Human+Machine Skill Development

The skill library was developed through a leak-safe learning loop. In `learn` mode, the solver first runs a strict no-gold solve and publishes a prediction. Only after publication does the learning path unlock gold, diagnose the prediction-vs-gold gap against a fixed failure taxonomy, and emit `reflection.md` plus `proposed-skill-updates.md`. A human reviewer decides what, if anything, graduates into a skill.

The loop is designed to distill rather than memorize. It forbids promoting task IDs, exact gold values, gold-only facts, or one-off wording hacks. Generalizable lessons — join defaults, row-grain warnings, tie policies, normalization rules, and abstention criteria — may be promoted. This gives the system a transparent, auditable capability layer without fine-tuning model weights.

This is the key methodological claim: a comparatively modest model becomes more capable when wrapped in the right agentic loop. The durable improvement is not a single longer prompt or a stronger hosted model; it is the repeated conversion of failure into readable skills, phase-specific prompts, and verification rules that future runs can reuse.

4.5 Contract-Gated Deterministic Publishing

Every answer is mediated by an `answer-contract.yaml` that records row grain, literal bindings, predicates, projections, output columns, and per-column status. The generated solution may produce a raw prediction, but the publisher emits only columns marked `supported`. Unsupported, risky, or abstained columns are dropped; if no supported output remains, the system publishes a narrow abstention rather than speculative extra columns.

This matters for the benchmark and for real analytical work. KDD scoring penalizes extra predicted columns, and real users need to know which parts of an answer are grounded. PiTrace's publish gate does not prove that every supported answer is correct, but it does structurally block unsupported columns from becoming the final answer.

4.6 Modality-Agnostic Evidence

PiTrace treats modality as a property of the workspace, not a separate product mode. Uploaded context is organized into CSV, JSON, database, document, video, and image folders and exposed through one workbench.

For non-tabular sources, the discovery phase normalizes evidence into text and structured artifacts that the same reasoning loop can use. Video handling samples frames with `ffmpeg`, drops near-duplicates, runs frame OCR through a vision model, and can transcribe audio; the result is a `video-context.md` artifact with per-frame text, table rows, and a timestamped transcript. Narrative documents are searched through prose-fact skills that recover candidate relations from text.

We do not claim general native video reasoning. Multimodal support here means extracting useful evidence from media into inspectable artifacts, then applying the same contract/evidence/plan/publish loop used for structured data.

4.7 A Better Loop, Not a Bigger Model

We used Qwen 3.5/3.6-family open-weights models behind OpenAI-compatible endpoints. Phase 2 solver runs used `Qwen3.6-35B-A3B`; other development and app runs used closely related Qwen 35B-A3B variants through OpenRouter and local LlamaCpp. Model name, endpoint, and key are configured through environment variables, so the same loop can run against local or hosted providers and stronger compatible models without changing solver code. The contribution is not a model upgrade. It directly addresses the rubric's warning that simply using a more expensive model should not receive a high score: PiTrace extracts reliability from phase structure, skills, tools, artifacts, review, repair, memory, and publishing.

4.8 Two Pi-Native Engines, One Control Plane

The engineering contribution is not only the visible workbench; it is the way the team specialized the Pi.dev framework twice inside one coherent system. Both engines run through the same embedded Pi SDK worker in `solvers/pi/src/pi-phase-worker.ts`. The worker registers the configured model provider, loads project-local `.pi` resources, filters skills according to `phase_config`, binds extensions, opens the correct session tree, streams tool events, writes phase logs, and records token usage. The engines differ because the runner gives each phase a different prompt, skill set, tool surface, and artifact contract.

Discovery engine. App surface: `/session/:id/discover`, discovery graph, source inspector, findings walk. Pi resources: `/discover`, `discovery-orientation`, optional `structured-data-analysis` and `video-context`. Tool surface: `kdd_discovery_emit` plus Pi read/bash tools, with media-profiler artifacts prepared before the phase. Durable output: `discovery/stream.jsonl`, `discovery-summary.md`, `graph.yaml`, `passages.jsonl`, `media-tables.jsonl`, and canonical `discovery-map.json`.

Answer engine. App surface: `/session/:id/answer`, answer thread, walls, feedback, prediction viewer. Pi resources: `/solve`, `/repair-solve`, `/review-solution`, `/feedback-repair`, `/learn-with-gold`, and the answer/review/modality skills. Tool surface: `kdd_discovery_query`, `kdd_request_human`, `kdd_prose_relation`, `kdd_prose_adjudicate`, and app-only `kdd_solve_emit`. Durable output: `answer-frame.md`, `answer-contract.yaml`,

`evidence-map.md`, `plan.md`, `solution.py`, `solution-review.md`, and `published.csv`.

The discovery engine is question-blind by design. The app creates a file-backed workspace, classifies uploads into context folders, and launches `npm run discover -- task_session`. The discover phase must describe every source, then emit relationships, groups, concerns, and synthesis events. Each `kdd_discovery_emit` call is a typed graph update, not a chat message. The server tails the resulting JSONL stream over SSE, while the compiler converts the same stream into reusable artifacts for later phases. A canonical node-id map keeps graph nodes stable across discovery and answering, so the solve overlay can recruit the exact sources the discovery graph already placed.

The answer engine starts only after a question exists. It preserves the discovery artifacts, injects the question and effective memory into an isolated answer root, and launches `npm run task` or `npm run feedback`. The solve phase first reframes the question against the discovered source graph, then writes an answer frame, contract, evidence map, plan, and deterministic Python. Mechanical execution, raw prediction validation, independent no-gold review, focused repair, and publishing are runner phases around that Pi reasoning loop. Post-answer feedback continues the same answer root and branches repair from the accepted solve-session leaf rather than restarting from a blank conversation.

The split is important because the two engines optimize for different risks. Discovery optimizes for source coverage, modality normalization, relationship mapping, and early concern surfacing. Answering optimizes for literal question coverage, row grain, supported columns, deterministic implementation, independent review, and abstention when support is missing. They share evidence through artifacts and tools, but they do not share a vague transcript context. That makes the workbench legible: a user can inspect what the discovery engine believed about the workspace and then watch the answer engine recruit, question, or reject those discovered sources.

4.9 Thin Deterministic Harness, Profile-Driven Specialization

PiTrace's reliability rests on a strict separation between a *generalized harness* and *per-task specialization*, and on the fact that specialization is composed from data signals rather than written as task-specific code.

The harness is the TypeScript runner in `solvers/pi/src/cli.ts`. It is identical for every task and every modality and contains no task IDs, dataset names, or per-task branches. It owns only mechanical, deterministic responsibilities: task discovery, context profiling, phase sequencing, timeouts and repair caps, generated-code execution, no-gold validation, human-wall plumbing, and contract-gated publishing. It never reasons about a task; it orchestrates phases and enforces guardrails.

Specialization happens in one small module, `solvers/pi/src/phase-config.ts`. The deterministic profiler writes `context-profile.md`; the harness reads *factual* signals from it — whether the workspace contains structured sources, prose, documents, or extracted video context — and maps the tuple `(phase, phase kind, profile signals, human mode)` to a required-plus-optional skill set. A pure-CSV task and a multilingual-video task run the same harness but load different specialist skills: `prose-fact-search` is offered only when prose exists, `video-context` only when a `video-context.md` artifact is present, `operator-escalation` only when a human is in the loop, and modality skills the task cannot use are pruned, so the model is handed a task-appropriate subset rather than all 17 skills every run. Each phase — discover, solve, repair, independent review, feedback repair, and learning — draws its own core set from the same library.

This is the concrete meaning of "we write the analyst, not the analysis." Specialization is skill *selection over a reusable library*, decided by deterministic profile signals — not per-task code generation and not a longer hidden prompt. A reviewer can confirm the entire selection policy in one file and verify that task identity never enters it. The payoff is both correctness — each phase sees only the decision procedures relevant to its inputs — and reproducibility: the same profile yields the same skill set on every run.

5. Evaluation and Evidence

We evaluate PiTrace along the Creative Track dimensions: end-to-end agent capability, human-agent collaboration, evidence quality, real-world practicality, and reproducibility. The evidence has three parts: validation over the full 111-task KDD Phase 1/Phase 2 input pool (51 Phase 1 and 60 Phase 2), seeded interactive workspaces that demonstrate the workbench, and a supplementary release of selected traces from `solvers/pi/solve-traces`. The trace bundle is released as per-folder zip archives at `https://tinyurl.com/team1286-traces`, so reviewers can inspect representative contracts, evidence maps, generated `solution.py` files, logs, reviews, repair diffs, publish reports, and final `published.csv` files rather than relying only on summarized claims. We make no leaderboard claim.

Metric	Evidence
Skills	17 task-agnostic skills, about 2,500 lines, no public task IDs, memorized answers, or hard-coded publish columns
KDD pool	All 111 KDD-provided Phase 1/Phase 2 input tasks (51 Phase 1, 60 Phase 2)
Traces	Representative subset of solver traces: `phase_1.zip`, `phase_2.zip`, and `custom.zip` from `solvers/pi/solve-traces` at `https://tinyurl.com/team1286-traces`
Sources	CSV 97, JSON 87, SQLite 85, Markdown/prose 111, PDF/doc 56, video 30
Repairs	19 KDD task roots with repair markers, plus the custom PBL feedback-repair trace
Modalities	CSV, JSON, SQLite, Markdown prose, medical PBL prose, video-derived context, Chinese-language records
Demos	Seeded autonomous, medical PBL, and multimodal/multilingual workspaces
Model	Open-weights Qwen 35B-A3B-family models; Phase 2 PiTrace runs use `Qwen3.6-35B-A3B` behind a swappable OpenAI-compatible endpoint served through OpenRouter or local LlamaCpp
Latency	About 6 minutes for a clean solve; about 11 minutes with one repair on one local workstation
Tokens	Appendix J reports usage as tokens from the `costs.jsonl` files included in the public trace archives

Case A — Scenario Specialization Through Workspace Learning

The medical PBL workspace is the strongest real-world-fit example in the submission. It uses a chronic coronary disease education module organized like a professional teaching case: an introduction, a knowledge guide, therapy appendices, and three patient narratives. The work is not table lookup. It is the kind of prose-heavy reasoning clinicians and medical learners practice in PBL: understand the local guideline frame, apply it to an individual patient, separate known facts from missing information, and explain the management rationale.

PiTrace adapts to this scenario from the workspace itself, not from a hard-coded medical pipeline. Discovery separates the guideline/reference corpus from the patient case collection and surfaces the main difficulty: open-ended clinical questions with no explicit schema, row grain, or output columns. The answer loop then uses the same general machinery used on other tasks: prose evidence search, answer contracts, deterministic execution, review, repair, human clarification, and contract-gated publishing.

This is not clinical validation or medical-device readiness. The claim is that PiTrace can ingest an unfamiliar professional education module, specialize from prose, and produce reviewable structured outputs in a realistic teaching and

decision-support workflow. Appendix D gives the scenario setup and solve traces in detail.

Case B — Autonomous End-to-End Solve
The Phase 1 `task_163` workspace demonstrates the fully automated path: profile -> solve -> run -> validate -> review -> publish. It combines CSV, JSON, SQLite, and Markdown and asks for approved expense types and total approved value for the *October Meeting* event. The agent resolves the join `expense.csv.link_to_budget -> budget.json.budget_id -> event.db.event_id`, binds "type of expenses" to `budget.category`, binds "total value approved" to `SUM(cost WHERE approved='true')`, cross-checks against `budget.spent`, and publishes exactly two supported columns: `type` and `total_value`.

Case C — Human-Assisted Correction and Repair
This case demonstrates the collaboration thesis. In one repair example, the agent initially interpreted a question as asking for events with the lowest total event cost. A human supplied one sentence of feedback: the task was asking for events with lowest-cost items. The solver did not restart from scratch; it continued from the existing answer context, repaired the interpretation, regenerated the deterministic code, and produced the corrected output.

A second Phase 1 repair example, `task_344`, asked how many male patients with normal white blood cell levels have abnormal fibrinogen. A focused feedback/review cycle expanded the male population by parsing additional patients described in `Patient.md`, changing the answer from 1 to 2. The final review accepted the delta after checking the broader population, the lab predicates, and the output contract. These examples show the intended human role: not babysitting low-level aggregation, but injecting judgment where the agent's interpretation is wrong or incomplete.

Case D — Multimodal and Multilingual Workspace
The Phase 2 `task_11` seeded workspace asks: *"According to the Large Subscription Fulfillment Standard configured in the video, which shareholder names meet both channel requirements?"* The key rule exists in a briefing video; the discovery pipeline extracts frames, OCRs the threshold panel into `video-context.md`, and the agent binds the two-channel predicate `ActualShares > 500000` and `OughtShares > 500000`. The shareholder records are Chinese-language, so the case is multimodal and multilingual at the same time. The agent considers all 220 records and publishes 88 qualifying shareholder names. The workbench shows the discovery graph, video/media panel, transcript/frame evidence, answer thread, and final prediction.

Baselines and Ablations
We use simple baselines to motivate the design. A one-shot LLM answer has no contract, evidence map, deterministic code, validation, repair, or publish gate. A CLI-only solver produces artifacts but loses the shared, correctable human-agent workspace. PiTrace without walls and feedback removes the main human judgment path shown in Case C. PiTrace without review and repair is faster but weaker at recovery.

The measured resource profile is token-based rather than dollar-based. We ran the submitted system with OpenAI-compatible Qwen endpoints served through OpenRouter and local LlamaCpp, so dollar spend is endpoint-dependent and not a stable system property. Appendix J reports the canonical `costs.jsonl` token records by phase, source group, and task.

The model comparison point is therefore architectural rather than a leaderboard claim. The OpenAI-compatible interface makes the model provider swappable, but PiTrace's argument is that reusable skills, prompted phases, deterministic execution, and bounded repair make a smaller open-weights model solve more reliably than a one-shot call to a stronger model without those controls.

Appendix C lists how each quantitative and case claim is supported in the submitted materials.

6. Real-World Considerations

Token use and latency. Local LlamaCpp runs have effectively zero marginal dollar cost on owned hardware, while OpenRouter runs have provider-specific pricing, so token consumption is the portable resource measure. Observed end-to-end latency on one local workstation is about 6 minutes for a clean single-cycle solve and about 11 minutes when one repair cycle fires. Phase timeouts, a capped repair count, quick-vs-deep modes, and streaming progress keep runs bounded and observable; Appendix J gives the token-cost breakdown.

Privacy and security. The app and solver operate on local workspace files. Generated `solution.py` is checked before execution for forbidden network, LLM, Pi, shell, package-installer, gold, and official-output access patterns and runs only deterministic local processing. Model credentials are passed through environment variables and are not written into `run-context.yaml`. Source-preview routes validate folder and filename inputs against allowlists and serve bounded previews. Normal solve phases are forbidden from reading gold answers or official output paths.

Scalability. Each workspace is a file-backed session root. Each answer gets an isolated answer root with copied input, artifacts, output, and event stream. The solver uses task-scoped workspaces and process-scoped environment variables, avoiding shared mutable global state between runs. File-backed state keeps the deployment barrier low and makes artifacts easy to inspect.

Human intervention. The system supports a spectrum: fully automated solving when the task is clear, agent-led/human-assisted solving through walls, post-answer feedback for repair, and memory-guided future runs. The human is invited at decision points where domain judgment matters; the system still relies on validation, no-gold review, and publish gating because human input is not a substitute for verification.

7. Limitations and Failure Cases

PiTrace is a working research prototype, not a finished platform.

Discovery is agentic and broad, so graph labels, concerns, and grouping can vary across runs. This is acceptable for exploratory orientation but production use would benefit from tighter discovery objectives, a stable concern taxonomy, and stronger cross-run consistency.

The largest technical improvements would be richer content parsers and stronger agentic search over the discovered workspace. The current implementation deliberately keeps discovery relatively light: `context-profile.md` records source inventories, schemas, capped samples, knowledge hints, and prose candidates; the discovery compiler builds a typed graph, passages, and media rows; the answer loop then uses `kdd_discovery_query`, `kdd_prose_relation`, and `kdd_prose_adjudicate` to recover deeper evidence only when the question needs it. That balance reduces latency and avoids spending a full parse budget on every document, video, or table when a user may ask only one or two narrow questions. The cost is recall: under-parsed prose, weak table structure, or insufficient passage ranking can force repairs or missed evidence. Future work should make parsing progressive: preserve the cheap orientation pass, but add cached deep parsers and more agentic query planning that can expand from BM25 passages and graph nodes into targeted extraction, relation recovery, and coverage audits on demand.

Multimodal support is evidence extraction into text and structured artifacts, not general native video understanding. Frame OCR runs on the multimodal solver model by default (or a dedicated vision model when configured) and audio is transcribed by a separate speech-to-text path, but the answer loop itself reasons over the extracted text and structured artifacts rather than the raw media. The in-browser cited-frame highlight is a provenance aid, not a formal lineage proof.

The released trace subset intentionally includes failed runs. Six KDD task roots reached solution cycles but did not publish, typically because of a contract-status mismatch or a final-cycle runtime error. We report these failures rather than hide them; they are useful evidence for why validation, repair caps, and abstention/publish refusal matter.

The gold-supervised skill-development loop is offline and human-gated by design. It is not online self-modifying behavior, and the gold path is strictly separated from scored solves. Its quality depends on reviewer judgment and the diversity of tasks used to drive it.

Finally, the UI exposes rich information. Dense solve logs and artifact tabs can still overwhelm a user. The answer thread and artifact views are a strong starting point; future work should summarize lineage more aggressively and highlight only the evidence directly supporting the final answer.

8. Reproducibility and Disclosure

The submission bundle contains the app, solver, seeded demo workspaces, report, reproducibility guide, and disclosure form. Reviewers can inspect the system via the Docker submission artifact rather than reconstructing a development environment from source scripts. The expected submission archive is `creative_team1286_code.tar.gz`, containing the Docker image `creative_team1286:v1`; the reviewer-facing setup, configuration, run commands, expected outputs, and troubleshooting notes are documented in `docker/creative_team1286_guide.md`.

A reviewer can open a seeded workspace, inspect discovery, ask a question, watch live solver events, inspect the answer thread, provide feedback, resolve a wall, and review artifacts. The seeded demo workspaces include pre-computed artifacts so a fresh clone is inspectable without a live run; they are labeled as seeded snapshots, not as proof of execution at review time. The Docker image and reproducibility guide expose model endpoint, model name, API key, and input, output, log, and workspace locations through configuration rather than hard-coded paths.

Disclosure summary: the solver builds on the Pi coding-agent harness from Earendil Works (pi.dev) and runs open-weights Qwen 35B-A3B-family models through a swappable OpenAI-compatible endpoint, served either by OpenRouter or local LlamaCpp, with `Qwen3.6-35B-A3B` used for the Phase 2 PiTrace solver. Reported primary solver runs use that endpoint contract rather than frontier-model-specific features. Frame OCR uses the same solver model by default, or a dedicated vision model when one is configured; an optional local Whisper path performs audio transcription, with hosted audio fallback only when configured. Generated solutions use Python stdlib, `sqlite3`, and `pandas`. Runner-side media extraction uses `pdfplumber`, `pymupdf`, `faster-whisper`, and the `ffmpeg` binary. During solve, the system may call configured OpenAI-compatible model endpoints, but it does not query external data APIs, third-party knowledge bases, or web-search services.

Appendix A — Mapping to the Scoring Rubric

Dimension (weight)	Where addressed
A. Problem Value & Realism (15)	§1: analyst/auditor and clinician-educator users, messy multi-source data, human judgment gap; §5 Case A and Appendix D medical PBL workspace
B. Innovation & Technical Contribution (20)	§4.1 agent-emergent workbench; §4.2 and Appendix E feedback/memory control; §4.3 reusable skills; §4.4 human-gated learning; §4.5 publish gate; §4.8 profile-driven skill specialization; §5 Case A workspace learning
C. Data Agent System Capability (20)	§3 two-loop perceive/act/correct architecture; §4.8 thin harness over one skill library; §5 cases and repair evidence
D. Effectiveness & Evidence Quality (15)	§5 metrics, cases, baselines; Appendix C evidence manifest; Appendices D-G case details
E. Reliability & Real-World Considerations (15)	§4.5, §4.7, §6, §7, Appendix J: swappable model endpoint, token cost, privacy, validation, failure reporting
F. Completeness & Usability (10)	§3, §4.1, §4.2, §8, Appendix E: runnable workbench, state visibility, live walls, feedback continuation, three-scope memory
G. Reproducibility & Presentation Quality (5)	§8 commands and Docker; Appendix B third-party component inventory; Appendix C verification map; Appendix H configuration map; Appendix I detailed figures; Appendix J token-cost tables

Appendix B — Third-Party Component Inventory

This appendix is the full, version-pinned inventory of third-party components. The condensed one-page **Disclosure Form** required by the submission is a separate deliverable, `creative_team1286_disclosure.md` (exported as `creative_team1286_disclosure.pdf`); this appendix is its detailed backing evidence. Every component below is used as-is, without modification. Versions are taken from committed manifests (`app/package.json`, `solvers/pi/package.json`, `solvers/pi/pyproject.toml`) and client CDN includes.

B.1 External Libraries, Frameworks, and Tools

Component	Library / Tool	Version	Acquired via	Role
App server	`fastify`	^5.3.2	npm	HTTP server / routing
App server	`@fastify/multipart`	^9.0.3	npm	File-upload handling
App server	`@fastify/static`	^8.1.0	npm	Static asset serving
App client	`cytoscape`	3.30.2	CDN	Discovery graph rendering
App client	`marked`	15.0.7	CDN	Markdown rendering
App client	`dompurify`	3.2.6	CDN	HTML sanitization
App client	`js-yaml`	4.1.0	CDN	YAML artifact parsing
App client	`papaparse`	5.5.3	CDN	CSV preview parsing
App client	`highlight.js`	11.10.0	CDN	Code/JSON/YAML syntax highlighting
App client	Google Fonts (Geist, Geist Mono, Newsreader)	—	CDN (`fonts.googleapis.com`)	Web typography
Agentic harness	`@earendil-works/pi-coding-agent`	^0.75.5	npm	Pi coding-agent SDK — Earendil Works (pi.dev); the solver runs as Pi sessions/extensions on this harness
Agentic harness	`@earendil-works/pi-ai`	^0.75.5	npm	Pi AI/provider utilities (pi.dev)
Solver runner	`typebox`	^1.1.38	npm	Schema definitions
Python dependency	`certifi`	>=2026.6.17	uv/PyPI	Certificate bundle dependency
Media extraction	`pdfplumber`	>=0.11	uv/PyPI	PDF text/table extraction
Media extraction	`pymupdf`	>=1.24	uv/PyPI	PDF/document processing
Media extraction	`faster-whisper`	>=1.0	uv/PyPI optional extra	Local timestamped speech-to-text for video audio
Generated solutions	`pandas`	>=2.2, <4	uv/PyPI	Tabular processing
Runtime	Node.js	>=22	system	Runner and app runtime
Runtime	Python	>=3.11, <3.15	uv-managed	Solution and media extraction runtime
System binary	`ffmpeg`	system PATH	system	Video frame sampling / audio extraction
System binary	`sqlite3`	system PATH	system	Database inspection
System binary	`uv`	system PATH	system	Python environment management

Generated `solution.py` is restricted to Python stdlib (`csv`, `json`, `pathlib`, `sqlite3`) plus `pandas`; it imports none of the media-extraction or network packages.

B.2 Pre-Trained Models

Purpose	Model	Default slug	Access	Notes
Main solver (text + image)	Open-weights Qwen 35B-A3B family; Phase 2 uses `Qwen3.6-35B-A3B`	`Qwen3.6-35B-A3B`	OpenAI-compatible endpoint, served through OpenRouter or local LlamaCpp	Registered as multimodal (`input: ["text","image"]`); fully swappable via `MODEL_NAME` / `MODEL_API_URL`

Purpose	Model	Default slug	Access	Notes
Frame OCR (vision)	Defaults to the main model above	`MODEL_NAME` (default)	OpenAI-compatible endpoint	Discovery-phase media extraction; a dedicated vision model (e.g. Qwen2.5-VL) may be set via `MEDIA_VISION_MODEL`, with `MEDIA_VISION_FALLBACK` as retry
Audio transcription	Whisper via `faster-whisper`	`small`	Local/offline model cache	Preferred path for timestamped audio transcripts
Audio transcription fallback	Hosted STT/audio model	`openai/whisper-1`, then `openai/gpt-audio-mini`	OpenAI-compatible endpoint	Used only when local STT is disabled or unavailable

No embeddings, fine-tuned, or locally trained weights are used. The main solver and frame OCR are accessed through configurable OpenAI-compatible APIs. Reported Qwen runs used two serving paths, OpenRouter and local LlamaCpp, under the same endpoint contract; this keeps the solver swappable without changing code. Frame OCR uses the main multimodal model by default unless a dedicated vision model is configured. Video audio uses local Whisper-first transcription when available, with hosted audio fallbacks only if configured.

B.3 Data Sources

DataAgent-Bench-style task inputs only: local CSV, JSON, SQLite, Markdown, and seeded video/image workspace files. During solve, the system may call configured OpenAI-compatible model endpoints, but it does not query external data APIs, third-party knowledge bases, or web-search services.

B.4 Human-in-the-Loop Components

Wall resolution, post-answer feedback repair, three-scope memory editing (platform/repo-instance, workspace, question/task), and human-reviewed promotion of proposed skill updates in learn mode. No human edits the generated `solution.py` or the scored prediction directly.

B.5 Original Contribution

Our original contribution is the 17 task-agnostic Pi skills, 7 per-phase prompts, 7 project-local Pi extensions, TypeScript runner/phase tree, contract and publish gate, deterministic context and media profilers, gold-supervised learning loop, and browser workbench. We reuse no external solver or agent-orchestration project beyond the Pi SDK itself.

Appendix C — Evidence Manifest

C.1 Supplementary Solver Trace Release

In addition to the main report materials, we release a representative subset of solver traces at `https://tinyurl.com/team1286-traces`. Each top-level `solvers/pi/solve-traces` subfolder is compressed as its own zip file so reviewers can download only the evidence they want to inspect. These archives do not contain every validation run over the full KDD Phase 1/Phase 2 task pool; they are supplementary audit material. They preserve the same artifact layout used by the solver: `answer-contract.yaml`, `evidence-map.md`, `plan.md`, generated `solution.py`, phase logs, review decisions, repair diffs, `publish-report.md`, and `published.csv` where the run produced a publishable answer. The root `costs.jsonl` files used to generate Appendix J's token and runtime statistics are included in these same archives.

Archive	Local source folder	Contents and purpose
`phase_1.zip`	`solvers/pi/solve-traces/phase_1`	Phase 1 development and validation traces: 51 task roots, 58 logged phase calls in `costs.jsonl`, autonomous solves, failed/repairing runs, wall and feedback examples, and representative artifacts such as `task_163` and `task_344`.
`phase_2.zip`	`solvers/pi/solve-traces/phase_2`	Phase 2 solver traces: 20 task roots and 45 logged phase calls in `costs.jsonl`, including the multimodal/document extraction and multilingual examples used in the report, plus contracts, evidence maps, media artifacts, reviews, and publish reports.
`custom.zip`	`solvers/pi/solve-traces/custom`	Custom scenario traces, currently the medical PBL `task_pbl` run with wall resolution, feedback repair, review logs, repair diff, publish report, final `published.csv`, and 4 logged phase calls in `costs.jsonl`.

These traces are reviewer-facing verification material, not hidden leaderboard evidence. They are an audit trail for the system behavior summarized in the report: what the solver thought the contract was, what evidence it used, what task-specific deterministic code it generated, how review and repair fired, and exactly what deterministic output was published.

C.2 Claim-to-Evidence Map

Claim	Evidence submitted	How to verify	Notes / caveat
17 reusable skills, about 2,500 lines	Source tree under `solvers/pi/.pi/skills`	Count `SKILL.md` files and line totals	Skills contain no public task IDs, memorized answers, gold-derived facts, or hard-coded publish columns
Project-local prompts and extensions	`solvers/pi/.pi/prompts`, `solvers/pi/.pi/extensions`	Inspect committed directories	Shows Pi-native architecture rather than a long hidden runner prompt
Profile-driven skill specialization (§4.8)	`solvers/pi/src/phase-config.ts` and `context-profile.md` per run	Read `phaseResourceHints` / `phaseRuntimeConfig`; confirm skill sets change with profile signals and no task ID appears	Specialization is skill selection over a reusable library, ~200 lines, no per-task code
Deterministic generated solutions	Solver source and generated artifacts in seeded workspaces	Inspect `solution.py`, validation reports, and publish reports	Generated solutions must not call Pi, LLMs, network, installers, or shell
Full KDD validation corpus	Summary table in §5 and reproducibility guide notes	Confirm the KDD-provided task pool under `kdd_data/phase_1/input` (51 tasks) and `kdd_data/phase_2/input` (60 tasks); rerun representative tasks where available	Validation covered all 111 provided Phase 1/Phase 2 input tasks; the released `solve-traces` bundle is a representative subset for audit, not the full validation archive
Source coverage counts	§5 summary table	Count file modalities under the 111 KDD input task roots	Counts are full-pool input coverage measurements, not trace-subset counts
Repair-triggering traces	§5 summary table and representative case artifacts	Inspect repair markers in `solvers/pi/solve-traces/phase_1`, `solvers/pi/solve-traces/phase_2`, and custom PBL artifacts	Repair count is trace-release evidence, not a claim that the released traces are the full validation archive

Claim	Evidence submitted	How to verify	Notes / caveat
Case A medical PBL workspace learning	Standalone custom `task_pbl` trace and seeded/app PBL workspace answer; `task_2002` is the PBL output/gold identifier	Inspect `solvers/pi/solve-traces/custom/task_pbl` wall, feedback repair, review, and `published.csv`; inspect app PBL answer metadata, contract, review, and final prediction	Demonstrates scenario specialization from workspace prose, not clinical validation
Case B autonomous solve	Phase 1 `task_163` seeded workspace and solver artifacts	Inspect `solvers/pi/solve-traces/phase_1/task_163` contract, evidence map, review, and `published.csv`	Demonstrates clean no-human path
Case C human correction	Feedback artifacts, repair diff, before/after prediction screenshots or seeded trace	Inspect answer thread and repair artifacts	Signature human-agent collaboration example
Runtime memory control	Appendix E plus seeded app artifacts	Inspect workspace and answer `memory.md` in Phase 1 `task_163` (app seed label `P2 task_163 [Overview]`); inspect `context/effective-memory.md` and memory classification in Phase 2 `task_29` (app seed label `Task 29 [Live]`)	Memory guides later interpretation but does not become source evidence or bypass publish validation
Feedback continuation	`solvers/pi/solve-traces/phase_1/task_344` and Case C	Inspect `human/feedback-###.md`, `repair-diff.md`, `logs/feedback-repair.log`, `logs/review-feedback-decision.yaml`, and changed solution cycles	Feedback repair is demonstrated in solver trace artifacts; the current app seeds do not include a feedback continuation snapshot
Case D video/multilingual solve	Phase 2 `task_11` seeded multimodal workspace, media panel, `video-context.md`, final prediction	Open seeded workspace and inspect answer thread	Multimodal means extraction-to-text, not native video reasoning
Token use and latency numbers	Canonical `costs.jsonl` logs included in the public trace archives plus Appendix J token-usage tables and charts	Compare Appendix J totals with the archived `phase_1/costs.jsonl`, `phase_2/costs.jsonl`, and `custom/costs.jsonl` files	Token consumption is reported directly; dollar cost is endpoint-dependent
Seeded snapshots are clearly labeled	Seeded workspace directories and reproducibility statement	Open seeded workspace without rerunning; run one task live if desired	Snapshots are for inspection; live execution is separately supported
Third-party disclosure	Appendix B third-party component inventory and the standalone disclosure form	Compare with committed manifests	All listed components are used unmodified

Appendix D — Medical PBL Scenario and Solve Trace

The medical PBL workspace was included because it is a real-world-style specialization test rather than another benchmark table. The corpus is a chronic coronary disease module with an introduction, a comprehensive knowledge guide, two therapy appendices, a case index, and three patient case files: Manjeet, Gloria, and Jimmy. Its teaching flow mirrors problem-based learning: learners first absorb a focused clinical topic, then apply it to patient narratives, then discuss what information is missing and what management choices follow.

The discovery phase identified two working groups: a clinical guideline reference corpus (`knowledge.md`, the appendices, and `intro.md`) and a patient case collection (`case1.md`, `case2.md`, and `case3.md`). It also surfaced the main risk: every input is prose, and questions such as "What else would you want to know?" have no explicit schema, row grain, or output columns. That is precisely why this case is useful for the Creative Track: the problem resembles professional teaching and decision support more than a conventional benchmark join.

D.1 Manjeet Trace

The standalone custom `task_pbl` solver run asks: "For patient Manjeet, age 62, What else would you want to know?" The first attempted abstention was rejected because an empty CSV had no header columns. The solver then raised a material question-understanding wall asking whether it should publish a list of information items or a header-only abstention. The human selected `list_information_items`, so the solver produced one supported column, `information_item`, with rows for missing information domains grounded in the module. A later feedback repair asked for justifications. The final published rows include domains such as health literacy, depression/anxiety screening, vaccination status, physical environment, insurance details, non-alcohol substance use, systemic racism and cultural factors, falls and frailty, disease-process understanding, symptom recognition, food insecurity, language needs, and patient goals. Each row cites the knowledge guide and explains why the case narrative had not already answered it.

D.2 Gloria Trace

The seeded browser workspace `Problem Based Learning – Medical Cases` asks: "How would you manage Gloria's medications?" The answer loop frames the row grain as one medication-management recommendation for Gloria at the Part Two visit. The published CSV contains five supported columns: `medication`, `current_dose`, `action`, `recommendation`, and `guideline_reference`. The core medication rows recommend tapering/deprescribing bisoprolol, considering a candesartan dose reduction because of orthostatic hypotension and falls risk, continuing ASA, continuing atorvastatin, and continuing the COPD inhaler. The answer also includes general follow-up, education, and multidisciplinary-process rows. The independent review accepts the result as mechanically valid and semantically grounded, while explicitly noting that the supplementary `general` rows could be a row-count risk if an external scorer expected only the five named medications.

The important design result is that PiTrace did not add a medical solver, specialized clinical code, or task-specific branches. It applied the same general workbench loop used elsewhere: discover the workspace, select prose and document evidence skills, frame an answer contract, map claims to sources, generate deterministic local code, validate the output, run independent review, repair contract mismatches or under-explained evidence, and publish only supported columns. That makes the PBL run a scenario-based specialization example: the domain behavior emerges from the local corpus and reusable skills, not from hard-coded medical logic.

Appendix E — Memory, Feedback, and Human Control

PiTrace separates three ideas that are often collapsed into one chat transcript: live user engagement during a solve, a feedback run that continues an existing answer after publication, and durable memory that can guide later questions. The separation matters because each path has a different safety contract.

E.1 Live Engagement While Solving

Walls are agent-requested decisions. In auto or assisted mode, the solver can pause when it has a material ambiguity and ask the user to choose or confirm a bounded interpretation. Valid wall gates are question understanding, plan quality, and final prediction review. A wall request carries the decision needed, two or three concrete options, the agent's proposed default, concise evidence references, and the fallback if the user does not answer.

The seeded workspace `P1 Task 11 [Wall]` exists to show this path in the app; it is Phase 1 `task_11`, distinct from the Phase 2 video task with the same task number. The app uses the bridge human channel, writes the user's ruling into the answer run's `human/` directory, and resumes the paused solver. The ruling is also kept in the answer stream so the wall is visible in the same answer thread as the contract, evidence, review, and prediction. If the user chooses to pin the ruling, the app appends it to question/task memory so the same fork does not need to be re-litigated on the next related run.

Human wall input is guidance, not evidence. It may choose an interpretation, confirm a risk posture, or ask for abstention/repair, but the final answer still has to be supported by task files and accepted by validation, review, and the publish gate.

E.2 Feedback Continuation Runs

Feedback is user-initiated correction after an answer exists. It is implemented as a solver continuation, not as a new blank run and not as manual editing of the published CSV. The app calls Pi's feedback mode on the existing answer root; the solver records `human/feedback-###.md`, branches a focused `feedback-repair` from the current solve-session leaf, regenerates the affected contract, evidence, plan, and deterministic code, runs `review-feedback`, and republishes through the same contract gate.

One compact repair example uses a lowest-cost-items correction: the agent initially interpreted the question as lowest total event cost; the user clarified that the target was events with lowest-cost items; the solver repaired from the existing context, regenerated deterministic code, and changed the output. A second representative Phase 1 trace, `task_344`, shows the same loop for a patient-count error caused by under-parsing prose records.

Feedback can also become memory, but only by deliberate pinning or editing. The immediate feedback run repairs the current answer; memory decides whether the lesson should affect future answers.

E.3 Three-Scope Runtime Memory

The app turns the solver's local feedback mechanisms into three persistent Markdown memory scopes:

Scope	Stored as	Applies to	Typical contents
Platform	Platform-wide app memory; implemented as repo-instance memory (`repo.md`)	All workspaces and all questions served by this app instance	Organization conventions, recurring interpretation preferences, global analysis rules
Workspace	`memory.md` in the workspace/session root	All questions asked against one uploaded corpus	Dataset-specific assumptions, known join conventions, domain vocabulary, recurring source caveats
Question/task	`memory.md` in one answer root	One question or answer thread, optionally inherited by a follow-up	Wall rulings, focused corrections, task-specific interpretation notes

Before an answer run starts, and again before a feedback continuation resumes, the app compiles the non-empty scopes into `context/effective-memory.md` with separate headings for platform/repo-instance, workspace, and question memory. The solver registers that file as a `memory` artifact and exposes it in `run-context.yaml` as effective runtime memory. The context profiler classifies it as memory rather than ordinary prose or knowledge, so it is not accidentally treated as source data.

This gives memory a narrow role: it improves interpretation and avoids repeating known mistakes, while leaving evidentiary burden unchanged. A remembered rule can tell the solver where to look or which ambiguity the human resolved before; it cannot supply unsupported answer values, bypass no-gold restrictions, add unsupported publish columns, or override the deterministic `solution.py` and `published.csv` validation path.

Appendix F — Skill Library Snapshot

The reusable capability layer is the skill library under `solvers/pi/.pi/skills`. Its purpose is to avoid overfitting the agent to a particular task question, demo workspace, or scenario. Each skill captures a general analytical behavior that should transfer across task families; none is allowed to encode public task IDs, memorized answers, gold-derived facts, or hard-coded publish columns. The current library contains these 17 task-agnostic skills:

Skill	Purpose
`aggregation-and-ties`	Aggregation, extremum, ranking, and tie handling rules
`answer-contracts`	Supported-column contracts, publishability, and abstention policy
`answer-shape-framing`	Row grain, singleton/list/table shape, and output column framing
`discovery-index-query`	Querying discovered source indexes and relationships
`discovery-orientation`	Question-blind workspace profiling and source orientation
`document-evidence`	Extracting and citing facts from documents
`evidence-mapping`	Mapping each output claim to source evidence
`join-planning`	Join keys, entity resolution, and multi-source relationship planning
`learning-reflection`	Post-gold failure diagnosis and generalizable lesson extraction
`metric-grain`	Metric definitions, denominators, population, and row-grain checks
`no-gold-review`	Independent review without reading gold answers
`operator-escalation`	Human wall criteria, bounded choices, and default behavior
`prose-fact-search`	Candidate fact recovery from narrative or semi-structured prose
`question-interpretation`	Literal binding, ambiguity tracking, and semantic framing
`structured-data-analysis`	Table/database inspection and deterministic analysis planning
`value-normalization`	Units, dates, labels, casing, nulls, and categorical normalization
`video-context`	Use of extracted frame OCR and audio transcript evidence

The table above is a catalog; the more important point for evaluation is how the skills map to general Data Agent capabilities:

General capability	Representative skills	Evaluator-visible artifacts / behavior
Understand the ask without overfitting to wording	`question-interpretation`, `answer-shape-framing`, `answer-contracts`	`answer-frame.md`, `answer-contract.yaml`, supported-column publish policy
Plan data work across heterogeneous sources	`structured-data-analysis`, `discovery-index-query`, `join-planning`, `metric-grain`	Discovery graph, source recruitment, deterministic join/filter/metric plan
Ground claims in inspectable evidence	`evidence-mapping`, `document-evidence`, `prose-fact-search`, `video-context`	`evidence-map.md`, cited document/video/prose evidence, Case A and Case D
Prevent common scorer-visible mistakes	`aggregation-and-ties`, `value-normalization`, `answer-contracts`	Narrow `published.csv`, normalized values, explicit tie and row-grain policy
Recover or ask for judgment when support is insufficient	`no-gold-review`, `operator-escalation`, `learning-reflection`	Review decisions, wall artifacts, feedback repair, human-reviewed proposed skill updates

The runner does not stuff all skills into every phase. `phase-config.ts` selects required and optional skills from deterministic profile signals: structured sources, prose/document presence, video context, repair mode, learning mode, and human mode. This is why a pure CSV task, a medical-prose task, and a video-derived Chinese-language task can share the same harness while seeing different specialist guidance.

One representative appendix-worthy skill is `operator-escalation`: it defines when the agent should stop and ask for human judgment, what evidence must be shown with the request, how many options are allowed, and how the solver should continue if the user does not respond. It connects the skills architecture to the workbench UX: the agent's uncertainty becomes an explicit wall artifact instead of an invisible hesitation inside a transcript.

Appendix G — Seeded Demo Workspaces

The Docker/app bundle includes seeded snapshots and wall traces so judges can inspect workflows without waiting for a live run. They are not hard-coded solver answers; they are app-session artifacts produced by the same workbench state model and labeled as seeded snapshots. The source task/phase column is the authoritative benchmark reference; app seed labels are the literal names shown in the browser seed picker.

Workspace	Source task/phase	Purpose	App seed label
Autonomous mixed-source solve	Phase 1 `task_163`	CSV + JSON + SQLite + Markdown, clean publish path	`P2 task_163 [Overview]`
Multimodal/multilingual video solve	Phase 2 `task_11` and Phase 2 `task_29`	Video frame/audio extraction plus Chinese-language records	`P2 task_11` and `Task 29 [Live]`
Medical PBL specialization	Custom PBL `task_pbl`	Professional prose module and patient-management prompt	`Problem Based Learning – Medical Cases`
Wall/human-intervention trace	Phase 1 `task_11`	Human bridge configuration and wall-capable run	`P1 Task 11 [Wall]`
Workspace/question memory	Phase 1 `task_163`	Reusable guidance for recurring joins and expense interpretation; workspace `memory.md` plus answer `memory.md`	`P2 task_163 [Overview]`
Compiled effective memory	Phase 2 `task_29`	Answer input includes `context/effective-memory.md`; profile and artifacts classify it as memory, not source evidence	`Task 29 [Live]`

The meeting discussion settled on keeping a small set of high-signal workspaces: the medical PBL case, task 29 for video and multilingual evidence, a wall case, and memory examples. This keeps the app inspectable while still showing the main claims: autonomous solving, media extraction, professional-domain prose, human walls, and reusable memory. Feedback continuation is documented through the committed solver trace for `task_344`, not through the current app seed set.

Appendix H — App and Solver Configuration Modes

The app exposes configuration that matters for both reproducibility and real operation.

Configuration	Default / behavior	Why it matters
`MODEL_API_URL`, `MODEL_API_KEY`, `MODEL_NAME`	OpenAI-compatible model endpoint	Lets the same solver run OpenRouter, local LlamaCpp, or another compatible model endpoint without code changes
`MODEL_REASONING=false`	Reasoning mode disabled in submitted runs	Keeps token accounting and phase behavior comparable across local and hosted endpoints
`MEDIA_VISION_MODEL`	Defaults to the solver model unless overridden	Allows separate frame-OCR model selection
`MEDIA_LOCAL_STT`, `MEDIA_LOCAL_STT_MODEL=small`	Local Whisper path for timestamped audio	Avoids relying on hosted audio when the Docker image has local weights
`MEDIA_AUDIO_MODEL`, `MEDIA_AUDIO_FALLBACK`	Hosted STT/audio fallback if configured	Lets audio degrade gracefully when local STT is unavailable
`humanMode=auto` for UI	Agent may raise a wall when materially confused	Supports human-in-control without forcing every task to pause
`humanMode=off` for eval	No blocking human wall during evaluation	Prevents unattended benchmark runs from waiting for input
`validationMode=deep`	Full validation/review path in seeded traces	Makes case artifacts more useful for inspection
`solutionMaxCycles` and phase timeouts	Bounded repair and solve loops	Keeps runs finite and failure modes visible
Theme, accent, and language UI settings	User-facing app preferences	Demonstrates app completeness; does not affect solver outputs

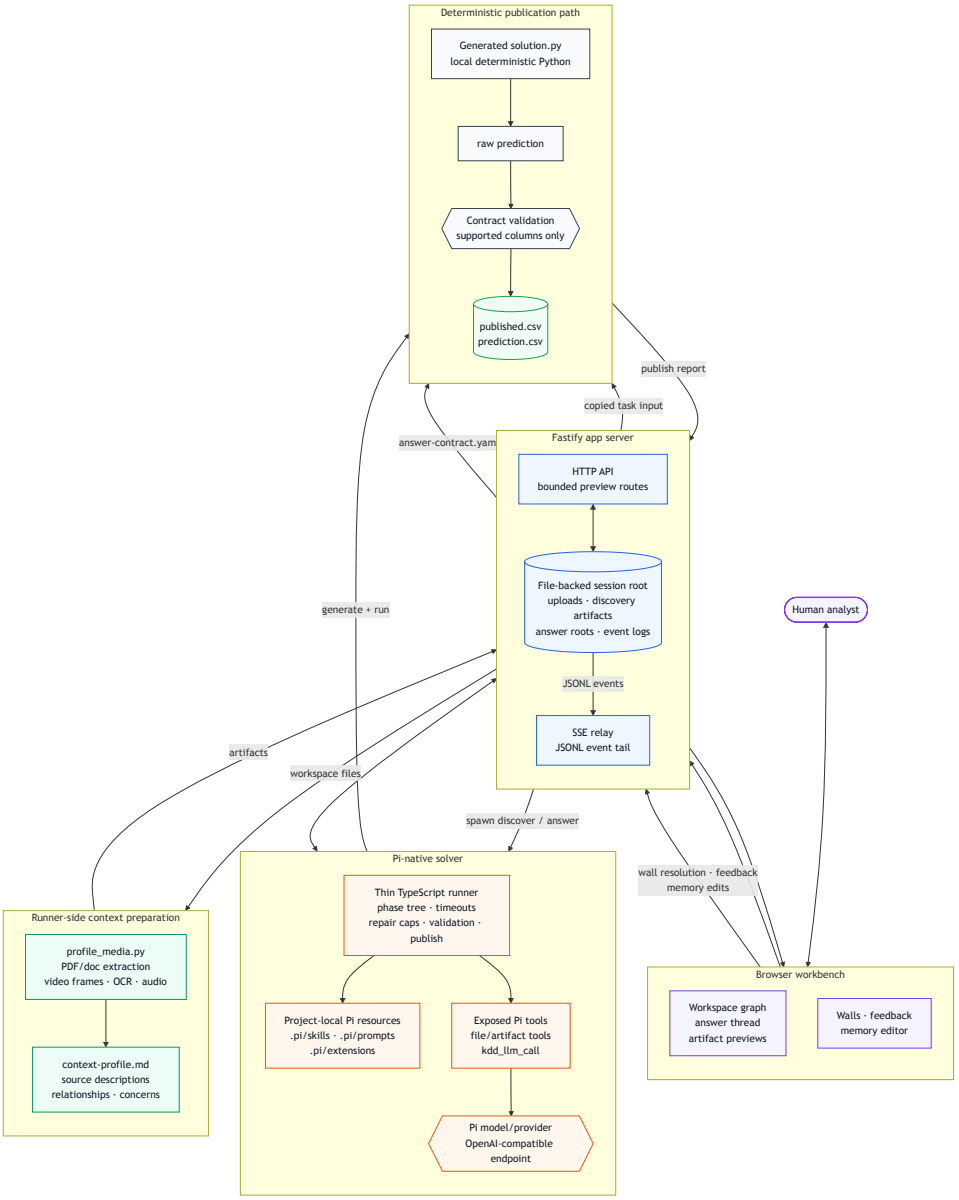
Media extraction is fail-open in the prototype: if PDF parsing, video OCR, or audio transcription fails, the run can continue with whatever artifacts were successfully produced. That is useful for benchmark robustness because a partial media failure should not necessarily block all tabular evidence. In production, the same condition should be surfaced more aggressively as a configuration or data-quality warning.

Appendix I — Detailed Architecture and Workflow Figures

The main report uses simplified figures for readability. The expanded diagrams below show the same system with implementation-level boundaries, artifact flow, and human-control paths.

I.1 Expanded End-to-End Architecture

PiTrace is organized as an observable workbench wrapped around a Pi-native solver, not as a hidden monolithic prompt. The browser and Fastify app own user interaction and file-backed session state; runner-side profiling converts mixed workspace media into concise artifacts; Pi resources, tools, and model context remain inside the solver boundary; and final prediction publication is constrained by deterministic local code plus a contract gate.

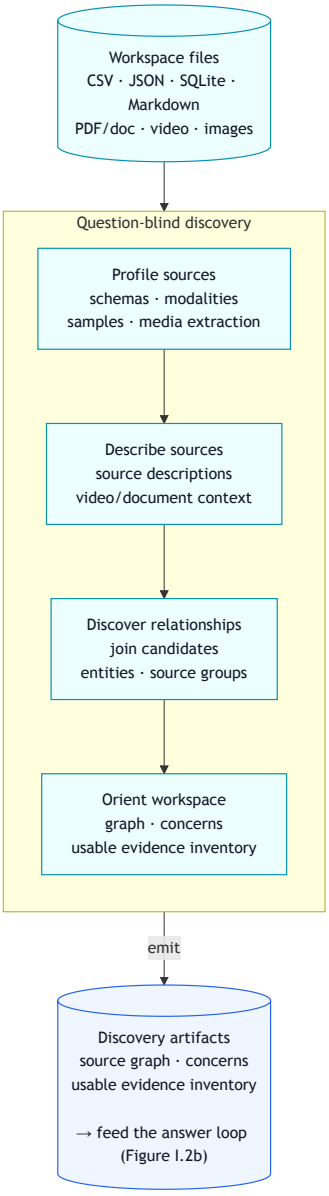


Detailed Figure I.1 — Expanded architecture with app events, session artifacts, media/context preparation, Pi resources, tools, model context, and publication gate.

Commentary. The figure depicts the end-to-end system boundary. Human actions enter through the workbench, the app records them as session artifacts and event logs, the runner invokes Pi phases with project-local skills/prompts/extensions, and only the deterministic publication path can produce `published.csv` or the scorer-visible `prediction.csv`.

I.2 Question-Blind Discovery Flow

The workflow is intentionally split into discovery and answering. Discovery is question-blind: it profiles the workspace once, records source descriptions, relationships, concerns, and usable evidence, then leaves those reusable artifacts for one or more later answer runs.

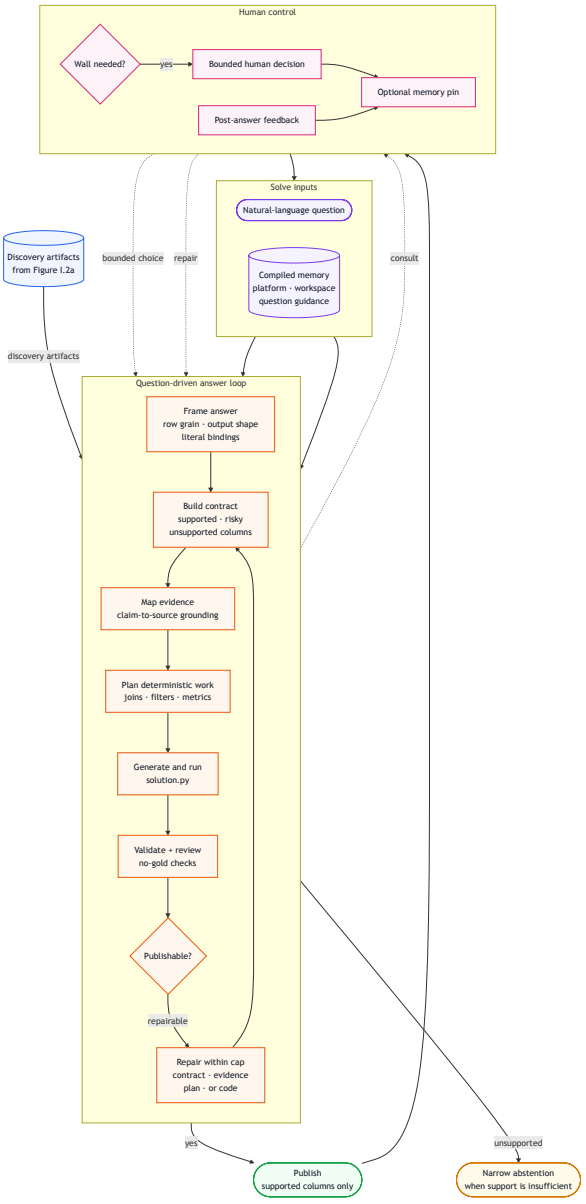


Detailed Figure I.2a — Question-blind discovery: profile the workspace, describe sources, discover relationships, orient the graph, and emit reusable discovery artifacts that seed the answer loop.

Commentary. This figure depicts the preparation pipeline before any specific question is interpreted. Its main point is separation of concerns: source and relationship understanding is reusable workspace context, while answer-specific choices are deferred until the next loop.

I.3 Question-Driven Answer Loop

The answer loop is contract-first and evidence-first. A natural-language question, compiled memory, and discovery artifacts are turned into a row grain, supported output contract, claim-to-source evidence map, deterministic execution plan, generated `solution.py`, validation/review result, and finally publish, repair, or abstention.



Detailed Figure I.2b — Question-driven answer loop: frame the answer, contract, map evidence, plan and run deterministic code, validate and review, then publish supported columns, repair within cap, or abstain. Human control (walls, feedback, memory pinning) attaches to the loop as dotted consultation paths.

Commentary. This figure depicts the solve-time control loop. Human walls, feedback, and memory pins can steer interpretation or request repair, but they do not bypass evidence mapping, deterministic code execution, validation, independent review, or the supported-column publish gate.

Appendix J — Token-Based Usage Cost

We report cost as token use, not dollars. The same solver can run against different OpenAI-compatible endpoints or local hardware, so dollar price is not a stable property of the system. The raw logs keep the original `cost_usd` field for provenance, but this appendix uses only the token and runtime fields. The source `costs.jsonl` files are included in the public trace archives described in Appendix C.

Each row in a `costs.jsonl` file is one **phase call**: one model call made by a phase such as `solve`, `review-solution`, `repair-solve`, or `discover`.

J.1 What Was Counted

This table defines the measurement boundary. It separates the logs that were counted from one discarded app-answer log, so the reader can see exactly what the appendix totals cover.

Item	Value
Cost logs audited	14
Cost logs counted	13
Phase calls counted	123
Logs skipped	1 discarded `.trash` app answer log
Distinct task labels	28
Model names in logs	3
Time span covered by logs	2026-06-20 22:02:52Z to 2026-07-04 00:04:09Z
Sum of logged phase runtimes	427.0 minutes
Failed phase calls	7 (5.7%)

The counted logs come from the cleaned solver traces (`solvers/pi/solve-traces/phase_1`, `phase_2`, and `custom`) plus seeded app workspace logs under `app/seed`. The skipped `.trash` log is listed in the audit table but is not included in totals.

J.2 How To Read The Token Columns

This table explains the token categories used throughout the appendix. The key distinction is between fresh tokens, which represent new model work, and cache-read tokens, which represent context reused from the endpoint cache.

Token measure	Meaning	Tokens	Share
Total tokens	All tokens reported by the model endpoint, including cached context reads	99,701,962	100.0%
Fresh tokens	New tokens for the call: input + output + cache writes	5,159,098	5.2%
Input tokens	Prompt/context tokens newly sent in the call	4,247,946	4.3%
Output tokens	Tokens generated by the model	911,152	0.9%
Cache-read tokens	Context tokens reused from the endpoint cache	94,542,864	94.8%

The main accounting point is that **99.7M total tokens does not mean 99.7M new prompt or output tokens**. Most reported usage is cached context reuse. The fresh-token total is 5.2M, while cache reads account for 94.5M tokens.

J.3 Main Takeaways

This table is the short interpretation layer for the appendix. It highlights the cost drivers without requiring the reader to inspect every detailed table.

Question	Answer
Where do most tokens go?	Solve and repair phases. Together, `solve`, `repair-solve`, and `feedback-repair` account for 72.7M total tokens.
How expensive are repair and feedback?	Repair/feedback phases account for 19 calls and 32.4M total tokens, or 32.5% of the logged total.
How large is a typical phase call?	The median phase call is 582,259 total tokens.
What is the 90th-percentile phase size?	1,891,322 total tokens. This means 90% of phase calls are at or below that size.
Where did failures occur?	All 7 failed phase calls are in the phase-one solver trace.
What was the largest single call?	`task_41` / `repair-solve`, with 4.43M total tokens.
What was the longest single call?	`task_199_mem` / `review-solution`, at 18.7 minutes.

J.4 Token Use By Phase

This table shows which solver phases consume the most model context. The main pattern is that solving and repair dominate total token use, while review is frequent but smaller per call.

Phase	Calls	Total tokens	Fresh tokens	Cache-read tokens	Cache-read share
solve	53	40,957,274	2,039,491	38,917,783	95.0%
repair-solve	11	22,647,101	1,048,375	21,598,726	95.4%
review-solution	44	18,366,517	1,191,952	17,174,565	93.5%
feedback-repair	4	9,130,723	543,109	8,587,614	94.1%
discover	5	7,337,674	216,189	7,121,485	97.1%
review-feedback	4	661,875	82,837	579,038	87.5%
learn-with-gold	1	582,259	27,015	555,244	95.4%
image-read-smoke	1	18,539	10,130	8,409	45.4%

Solve phases are large because they carry the question, discovered context, answer contract, evidence, generated code, validation output, and review history. Repair phases are also large because they reuse that context and add the correction or failure state.

J.5 Token Use By Trace Group

This table shows where the measured usage came from. Phase 1 carries the most runtime and all failed phase calls; Phase 2 covers more distinct tasks with no failed phase calls in these logs.

Trace group	Calls	Tasks	Total tokens	Fresh tokens	Runtime min	Failed calls
Phase 1 solver traces	58	9	48,165,206	2,383,400	216.7	7
Phase 2 solver traces	45	20	33,491,528	1,865,107	118.4	0
Seeded app answer logs	11	1	7,584,532	471,942	47.1	0
Seeded app discovery logs	5	1	7,337,674	216,189	32.9	0
Custom PBL trace	4	1	3,123,022	222,460	11.9	0

J.6 Highest-Use Tasks

This table identifies the tasks that drive the aggregate cost. The highest-use items are not just large tasks; they are tasks with repeated solve, review, repair, feedback, or app-session phases.

Task	Calls	Total tokens	Fresh tokens	Runtime min	Repair/feedback calls	Failed calls
`task_2002`	16	19,182,132	952,709	52.2	6	0
`task_session`	23	15,074,526	726,853	107.4	1	2
`task_199_mem`	7	12,158,026	481,350	51.1	2	2
`task_41`	4	9,033,403	286,435	19.2	2	0
`task_11`	10	6,693,284	368,187	26.4	1	1
`task_29`	6	3,193,450	209,654	24.1	0	1
`task_pbl`	4	3,123,022	222,460	11.9	2	0
`task_51`	4	2,664,141	151,688	7.8	1	0
`task_8`	2	2,649,326	100,976	9.7	0	0
`task_2001`	6	2,635,598	185,418	15.6	2	1

J.7 Model Mix

This table checks whether usage is concentrated in one model configuration. Most calls and tokens come from the submitted Qwen3.6 GGUF endpoint, while seeded app logs account for the smaller model-name variants.

Model	Calls	Total tokens	Fresh tokens	Cache-read share
`unsloth/Qwen3.6-35B-A3B-GGUF:BF16`	114	91,069,238	4,840,783	94.7%
`Qwen3.5-35B-A3B-Q4_K_M.gguf`	6	5,972,523	212,152	96.4%
`Qwen3.6-35B-A3B-UD-Q4_K_M.gguf`	3	2,660,201	106,163	96.0%

J.8 Figures

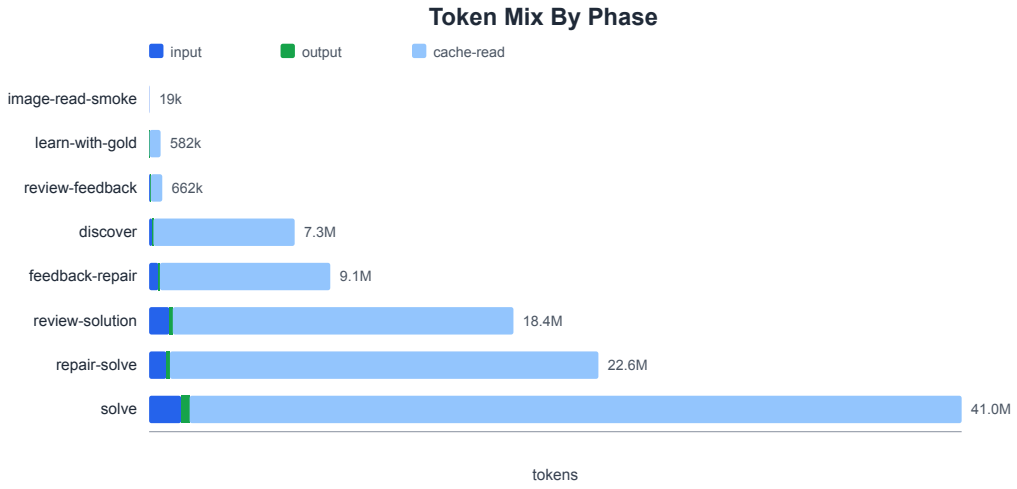


Figure J.1 — Token mix by phase. Each horizontal bar is one solver phase, split into fresh input, generated output, and cache-read context. The long cache-read segments show that the headline 99.7M-token total is mostly reused context, not new prompt or answer text; solve and repair phases dominate because they carry the full contract, evidence, code, validation, and review state.

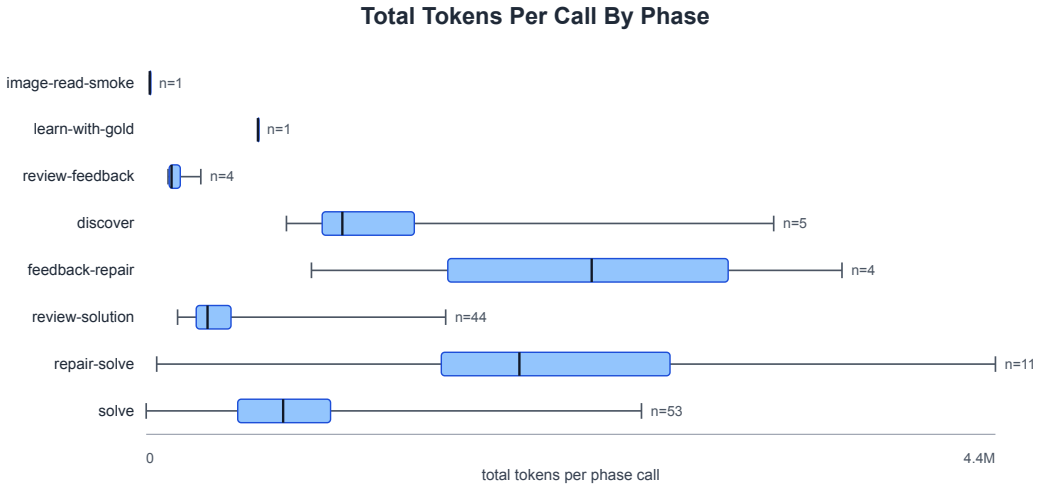


Figure J.2 — Total tokens per call by phase. The boxplots show the distribution of call sizes inside each phase type, with call counts annotated at the right. This makes the variance visible: review calls are frequent but usually compact, while repair and feedback calls can be larger because they include the prior failure state plus the revised contract, evidence, and code context.

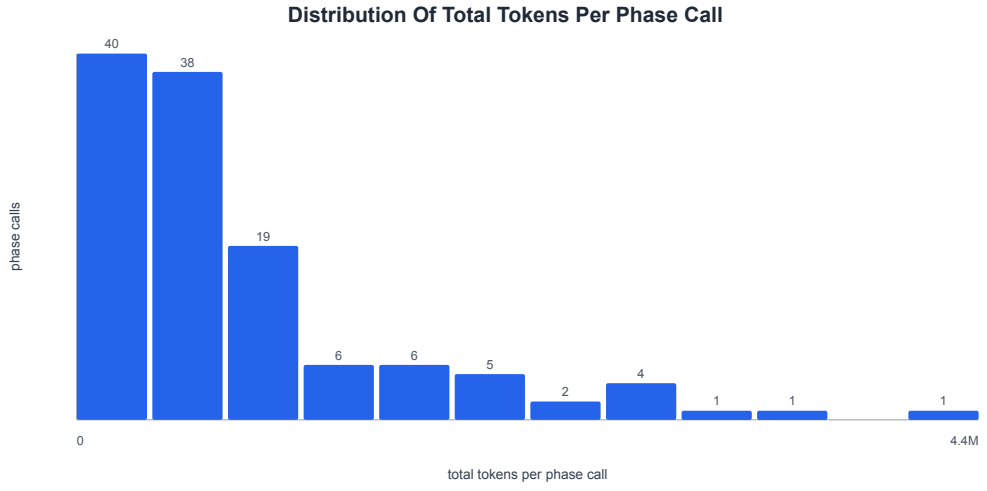


Figure J.3 — Distribution of total tokens per phase call. The histogram groups all counted phase calls by size so the long-tail behavior is visible. Most calls sit well below the largest outliers, which is why the appendix reports median and 90th-percentile phase sizes rather than treating the maximum as typical.

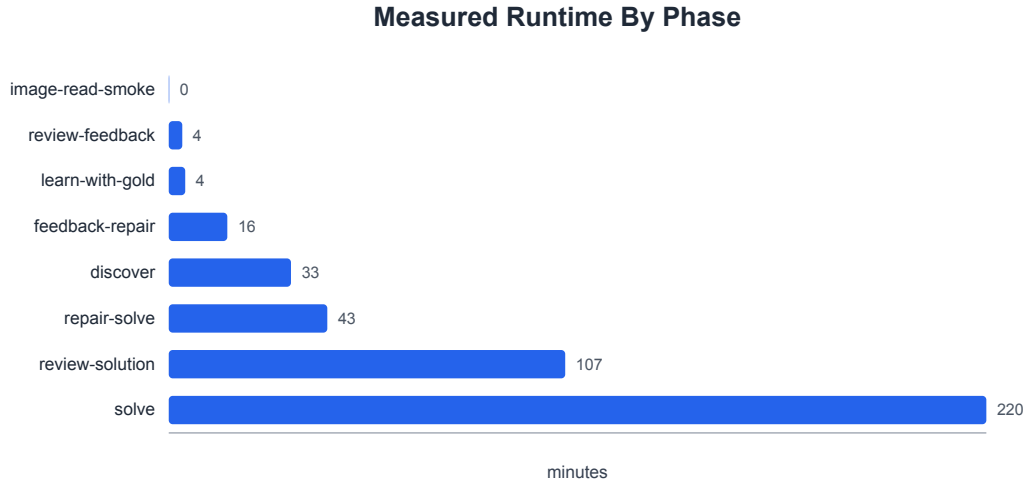


Figure J.4 — Measured runtime by phase. The bars aggregate logged wall-clock phase duration, not model price. Runtime follows the same broad pattern as token use: solve and review consume most elapsed phase time because they run often, while repair is less frequent but costly when it occurs.

Read together, the four figures separate different resource questions. Figure J.1 explains where aggregate token volume comes from, Figure J.2 shows how much individual calls vary by phase, Figure J.3 shows whether the largest calls are typical or tail events, and Figure J.4 checks whether elapsed runtime follows the same pattern as token use. The intent is to make the evaluation cost profile auditable without tying it to one provider's dollar pricing.

The figures in this section are regenerated directly from the counted `costs.jsonl` logs.